Algoritmi Paralleli e Tecniche di Analisi Algoritmica

Algoritmo: Borůvka Parallelo

Francesco Stabile

20 febbraio 2026

1 Introduzione

1.1 Descrizione dell'algoritmo

L'algoritmo di Borůvka[1][2][3] è un algoritmo greedy per determinare il *minimum spanning tree* di un grafo non orientato, pesato e connesso $G(V, E)$. L'algoritmo procede inizializzando ogni vertice del grafo come una componente connessa distinta. A ogni iterazione, per ogni componente, viene individuato l'arco di peso minimo che lo collega a una componente diversa. Nel caso di parità, una regola di spareggio viene utilizzata affinché non ci sia ambiguità e l'algoritmo sia deterministico. Tutti gli archi così selezionati vengono aggiunti alla soluzione, fondendo progressivamente le componenti connesse e diminuendone il numero ad ogni iterazione. Il processo viene ripetuto finché non rimane una singola componente connessa, la quale costituisce il *minimum spanning tree* cercato. Le immagini seguenti rappresentano alcuni passi chiave dell'algoritmo sequenziale:

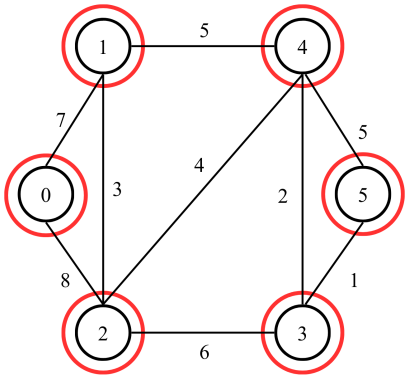


Figura 1: Primo passo dell'algoritmo, tutti i vertici sono componenti distinte

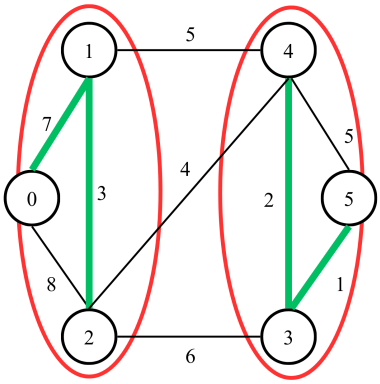


Figura 2: Per ciascuna componente, si individuano gli archi a peso minimo tra componenti distinte

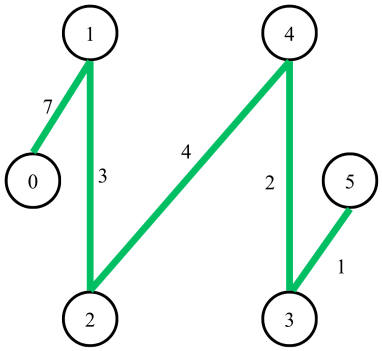


Figura 3: Passo finale, l'unica componente rimasta determina il *minimum spanning tree*

1.2 Pseudocodice

Algorithm 1 Borůvka sequenziale

```
1:  MST ← {V, ∅}
2:  components ← G.num_vertices
3:  while components > 1 do
4:      for i ← 1 to components do
5:          safe[i] ← NULL
6:      for all edge uv ∈ E do
7:          if comp(u) ≠ comp(v) then
8:              if safe[comp(u)] = NULL or w(uv) < w(safe[comp(u)]) then
9:                  safe[comp(u)] ← uv
10:             if safe[comp(v)] = NULL or w(uv) < w(safe[comp(v)]) then
11:                 safe[comp(v)] ← uv
12:     for i ← 1 to components do
13:         add safe[i] to MST
14:     components ← CountComponentsAndMap(MST)
15: return MST
```

1.3 Analisi dell'algoritmo sequenziale

Il ciclo `while` viene eseguito $O(\log V)$ volte, in quanto, ad ogni iterazione, il numero di componenti connesse si riduce almeno della metà. Questa riduzione è garantita dal fatto che ciascuna componente si fonde con almeno un'altra componente e nel peggiore dei casi si fondono a coppie. Il corpo del ciclo `while` impiega tempo $O(E)$, in quanto, ad ogni sua iterazione, si esamina ciascuno degli E archi del grafo per identificare gli archi *safe*, cioè quelli di peso minimo che connettono due componenti connesse distinte. Infatti, le restanti operazioni, come l'aggiunta degli archi al *minimum spanning tree* o le operazioni di assegnamento richiedono tempo costante $\Theta(1)$, in particolare il ciclo for per determinare gli archi *safe*, impiega $O(E)$ iterazioni, il ciclo for per il reset di *safe*[] o per l'aggiunta degli archi *safe*[] ad *MST*, svolge $O(V)$ iterazioni nel caso peggiore, e la procedura CountComponentsAndMap() su *MST* costa $O(V)$, in quanto un *minimum spanning tree* ha $V - 1$ archi e V vertici. Poiché l'input è per ipotesi un grafo connesso, abbiamo $V \leq E + 1$, dunque $O(V + E) = O(E)$, pertanto la complessità totale è pari a $O(E \log V)$.

2 Design Parallelo

2.1 Selezione dell'algoritmo: Borůvka - Prim - Kruskal

Per il problema del *minimum spanning tree*, le scelte possibili sono principalmente gli algoritmi di Borůvka, Prim[1] e Kruskal[1]. La selezione di Borůvka è giustificata dalla sua enorme potenzialità di parallelismo durante le prime iterazioni, mentre Prim o Kruskal offrono meno possibilità in quanto gran parte dell'esecuzione risulta sequenziale. In particolare, il fattore determinante che ha fatto sì che la scelta ricadesse su Borůvka è la possibilità di selezionare gli archi in modo indipendente per ciascuna componente, consentendo così un bilanciamento del carico di lavoro tra i processi, aspetto meno sfruttabile nelle altre due alternative. La possibilità di parallelismo riscontrata nell'algoritmo di Prim è solo durante l'aggiornamento dei pesi, poiché la selezione del prossimo vertice da utilizzare dipende dalle scelte precedenti. Infine nell'algoritmo di Kruskal, sarebbe possibile parallelizzare l'ordinamento dei pesi, ma non le restanti fasi. Per tali motivazioni, Borůvka è stato scelto come candidato ideale con più potenzialità parallele riscontrate a confronto delle altre due alternative, ma rappresenta comunque grandi sfide nelle ultime iterazioni, quando le componenti connesse si riducono notevolmente.

2.2 Strategia proposta

Diverse strategie e ottimizzazioni sono state provate prima di raggiungere una scalabilità accettabile. La prima sfida da affrontare è stata la scelta della distribuzione del grafo ai vari processori, dunque la modalità con cui partizionare l'*input*. Sebbene inizialmente, suddividere i V vertici sembrava la scelta ottimale poiché era possibile ottenere indipendentemente l'arco minimo globale, senza alcuna comunicazione necessaria nelle prime iterazioni, da un'analisi più accurata si è preferito distribuire gli archi. Ogni processore è pertanto responsabile di un sottoinsieme di archi.
Il vantaggio principale fornito dalla distribuzione degli archi è un carico di lavoro più bilanciato, infatti distribuendo i vertici con i relativi archi incidenti, alcuni processi potrebbero avere molti più archi da processare pertanto l'*overhead* dovuto all'*idling* potrebbe aumentare. Questa partizione è stata mantenuta per ogni strategia proposta e di seguito raffigurata.

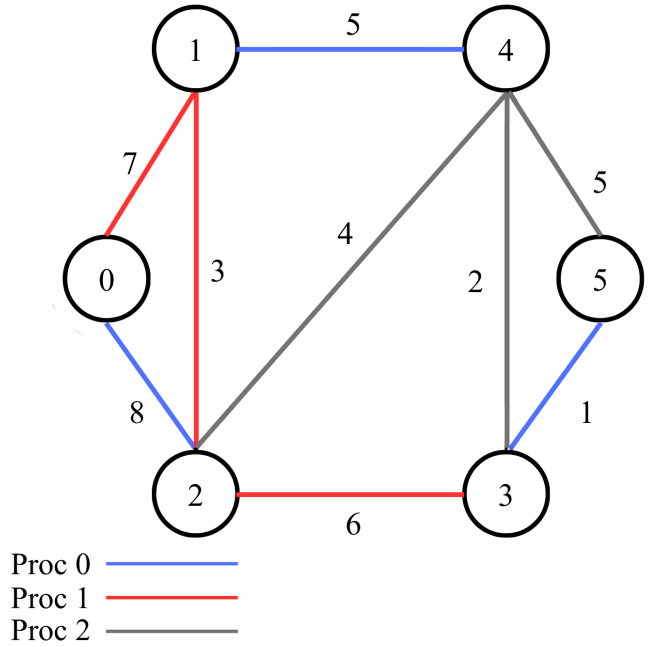


Figura 4: Esempio di *Scatter* degli archi con 3 processori

Per mantenere traccia delle componenti connesse durante le iterazioni dell'algoritmo, poiché ogni processore deve conoscere la componente di appartenenza di ciascun vertice, si è scelto di utilizzare una struttura dati *Union-Find*[4][5] replicata su ciascun processore, utilizzando per la funzione *find* e *union* rispettivamente, *path compression* e *union by rank* per ottimizzarne la complessità ed ottenere $O(\alpha(V))$, dove $\alpha(V)$ è la funzione inversa di Ackermann e può essere trattata come costante, ciò giustifica la scelta della struttura. Di seguito viene riportata l'implementazione utilizzata per la struttura *Union-Find*:

```
typedef struct UF {
    igraph_int_t rank;
    igraph_int_t parent;
} UnionFind;

//Inizializza la struttura
void init_union_find(UnionFind* uf, igraph_int_t
   n_vertex, igraph_int_t* num_components) {
    for (int i = 0; i < n_vertex; i++) {
```

```
        uf[i].parent = i;
        uf[i].rank = 0;
    }
    *num_components = n_vertex;
}

//Trova la componente di appartenenza di vertex
   risalendo la struttura e comprimendo i cammini
igraph_int_t find(UnionFind* uf, igraph_int_t vertex) {
    if(uf[vertex].parent == vertex){
        return vertex;
    }
    uf[vertex].parent = find(uf, uf[vertex].parent);
    return uf[vertex].parent;
}

//Fonde v1 con v2 e restituisce 1 in caso di fusione
   avvenuta, 0 altrimenti.
//Imposta inoltre num_components al nuovo valore dopo
   l'eventuale fusione.
int merge_components(UnionFind* uf, igraph_int_t v1,
   igraph_int_t v2, igraph_int_t* num_components) {
    int r1 = find(uf, v1);
    int r2 = find(uf, v2);
    if(r1 != r2){
        if(uf[r1].rank < uf[r2].rank){
            uf[r1].parent = r2;
        }
        else if(uf[r1].rank > uf[r2].rank){
            uf[r2].parent = r1;
        }
        else{
            uf[r2].parent = r1;
            uf[r1].rank += 1;
        }
        *num_components -= 1;
        return 1;
    }
    return 0;
}
```

1. AllReduce inefficiente

La prima strategia prevedeva la ricerca degli archi minimi locali per ciascuna componente vista da ogni processo, successivamente una comunicazione collettiva *AllReduce* con un operatore *custom* permette ad ogni processo di ottenerre l'arco minimo globale per componente, infine i vari processi applicano i *merge* nello stesso ordine per mantenere la *Union-Find* coerente e sincronizzata.

```
//Operatore custom per la comunicazione AllReduce
//Sceglie l'arco dal peso minimo e lo inserisce nel
   buffer di output
    void find_min_edge(void* edge_in, void* edge_inout,
       int* len, MPI_Datatype* dptr){
    Edge* e_in = (Edge*) edge_in;
    Edge* e_out = (Edge*) edge_inout;
    for (int i = 0; i < *len; i++) {
        if (e_in[i].weight < e_out[i].weight) {
            e_out[i] = e_in[i];
        }
    }
}
```

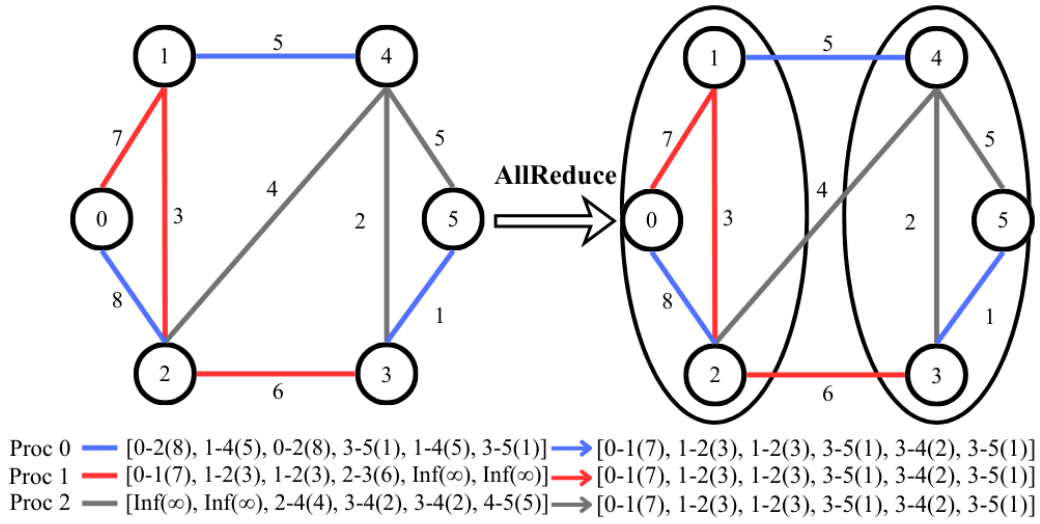


Figura 5: Esempio di una *AllReduce* con 3 processori

In questo primo tentativo però, il volume di dati scambiato con la *AllReduce* non scala con le componenti che nel caso peggiore dimezzano. Nel successivo approccio si è cercato di ottimizzare questo aspetto ottenendo dei discreti risultati.

2. Resizing AllReduce

L'ottimizzazione alla base di questa strategia è ridurre il peso di ogni *AllReduce* effettuata, infatti una volta eseguite le fusioni tra le componenti, quest'ultime si riducono notevolmente e le successive comunicazioni collettive avranno sempre minor impatto se eseguite sul numero di componenti attive rimaste. Per ottenere ciò è stato necessario introdurre una fase di *mapping* affinché tutti i processi indicizzassero le radici delle componenti allo stesso modo.

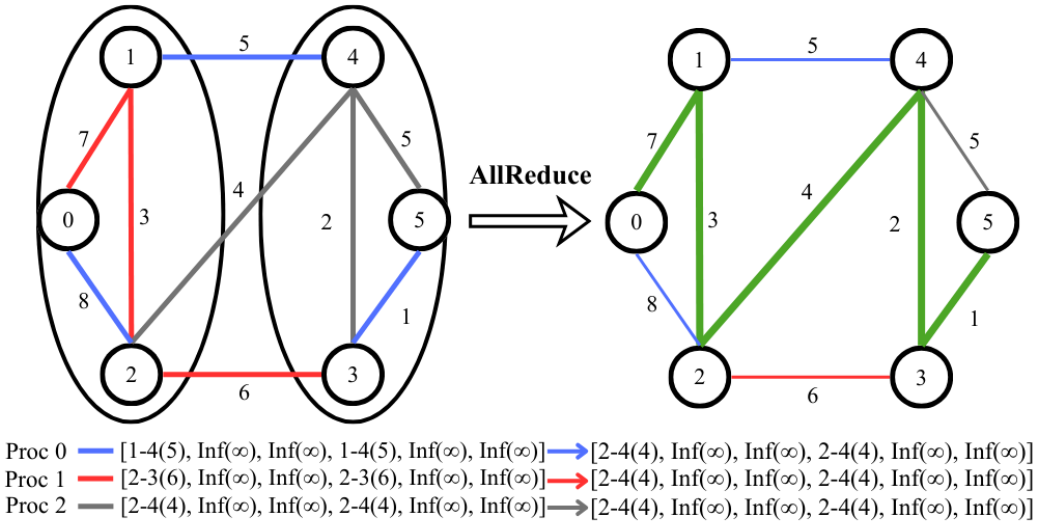


Figura 6: *AllReduce* effettuata nella prima strategia

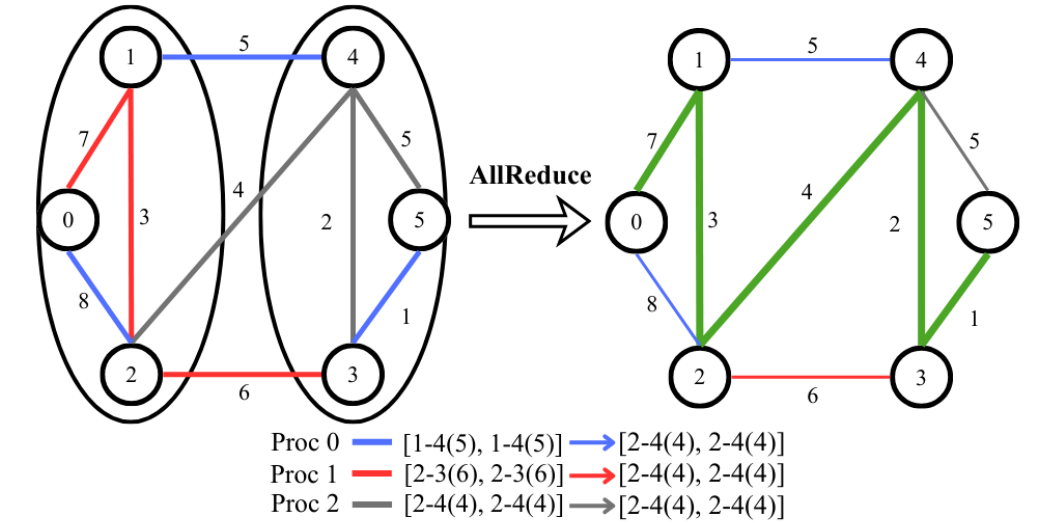


Figura 7: *AllReduce* ottimizzata nel secondo approccio

Con questa accortezza il peso delle comunicazioni viene ridotto sensibilmente, ma le prestazioni non sono ancora soddisfacenti, si ottiene infatti, uno

speedup di 1.7 con un'efficienza relativa di circa il 42%. La fase di ricerca degli archi minimi locali può essere ottimizzata ulteriormente, poichè già dopo la prima iterazione, molti archi di ciascun processore risultano interni alle componenti, pertanto controllarli nuovamente ad ogni iterazione accresce il tempo di esecuzione. Quest'idea ha portato alla versione finale dettagliata nel seguito.

3. Versione Finale

La presente idea proposta è identica alle precedenti, ma con delle ottimizzazioni che rendono possibile il raggiungimento di buone prestazioni. La ricerca degli archi minimi locali viene infatti ridotta ad ogni iterazione, al numero di archi effettivamente attivi. Un arco è considerato attivo nel momento in cui collega due vertici a due componenti distinte, così facendo non si processano più gli archi interni in maniera ridondante. Per ottenere ciò, è necessaria una fase di *cleanup* degli archi dopo le fusioni. Questa fase non richiede comunicazione e può essere svolta in parallelo, per velocizzare ulteriormente la fase di ricerca della successiva iterazione, si possono sovrascrivere i vertici incidenti degli archi locali di ciascun processo, con le nuove radici di appartenenza formate dalle fusioni, in tal modo non c'è bisogno di cercare la componente del vertice e la ricerca si riduce ad un semplice accesso alla memoria.

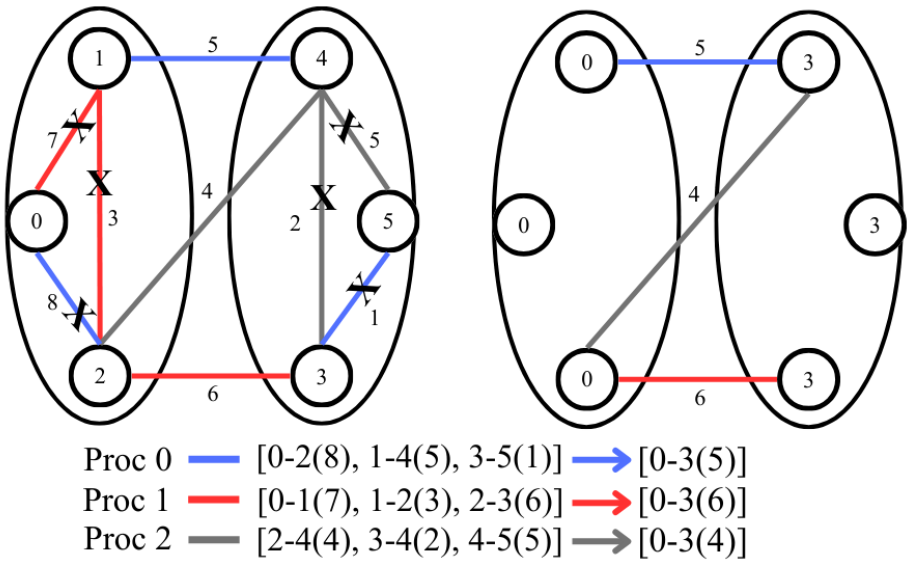


Figura 8: Esempio di *cleanup* con 3 processori,

Infine è stata modificata la funzione per la ricerca del minimo utilizzata per l'operazione di *AllReduce*, in quanto era sprovvista di una regola di spareggio esplicita nel caso di archi con peso minimo equivalente, di conseguenza in alcune casistiche veniva inficiata la correttezza dell'algoritmo, di seguito è riportata la nuova implementazione.

```
// Nuova implementazione con regola di spareggio
// Se il peso coincide, l'arco con il vertice sorgente
   minimo viene scelto
// Se il vertice sorgente coincide, l'arco con il
   vertice destinazione minimo viene scelto
void find_min_edge(void* edge_in, void* edge_inout, int*
   len, MPI_Datatype* dptr) {
    Edge* e_in = (Edge*) edge_in;
    Edge* e_out = (Edge*) edge_inout;
    for (int i = 0; i < *len; i++) {
        if (e_in[i].weight < e_out[i].weight) {
            e_out[i] = e_in[i];
        }
        else if (e_in[i].weight == e_out[i].weight &&
           e_in[i].weight < IGRAPH_INFINITY) {
            if (e_in[i].from < e_out[i].from) {
                e_out[i] = e_in[i];
            } else if (e_in[i].from == e_out[i].from &&
               e_in[i].to < e_out[i].to) {
                e_out[i] = e_in[i];
            }
        }
    }
}
```

3 Analisi della soluzione proposta

All'interno della presente sezione viene analizzata la complessità computazione e spaziale, altresì la relazione di isoefficienza e la funzione di scalabilità.

3.1 Complessità computazionale

L'analisi della complessità può essere suddivisa in quattro sezioni all'interno del `while` ed un ulteriore parte al di fuori del ciclo principale. Il calcolo delle variabili per la distribuzione degli archi impiega $\Theta(p)$, mentre l'inizializzazione delle strutture necessarie per il calcolo come `initUnionFind`, costa

$O(V)$. All'interno del ciclo che costa $O(\log V)$ iterazioni, troviamo quattro fasi elencate di seguito:

Mapping: Tutti e V vertici vengono scansionati per trovare la loro componente di appartenenza, una volta ottenuta, viene rieffettuata l'indicizzazione da 0 al numero di componenti attive -1, dunque impiega $O(V)$.

Search: La ricerca all'interno degli archi assegnati per ottenere i minimi locali, impiega $O(\frac{E}{p})$.

Comm: Comprende la comunicazione collettiva analizzata in seguito e la fusione delle componenti. Quest'ultima, impiega nel caso peggiore $O(V)$ fusioni ed ognuna di esse costa $O(\alpha(V))$, dove $\alpha(V)$ è la funzione inversa di Ackermann, pertanto si può concludere che il costo totale sia $O(V)$.

Cleanup: Vengono rimossi gli archi locali ormai fusi, si ricercano all'interno degli archi attivi, quelli interni alle componenti e si rimuovono, pertanto il costo totale è $O(\frac{E}{p})$.

Moltiplicando il costo di una singola iterazione per le $O(\log V)$ iterazioni otteniamo $O(\log V(\frac{E}{p} + V)) = O(\frac{E}{p} \log V + V \log V)$.

3.2 Complessità spaziale

Ciascun processore deve mantenere in memoria la *UnionFind* ed array di supporto per il *mapping* e la comunicazione, oltre agli archi assegnati, pertanto la complessità finale è $O(V + \frac{E}{p})$ per processore.

3.3 Complessità delle comunicazioni

Il costo della comunicazione, non considerando l'I/O, è dominato dalla unica operazione collettiva `AllReduce`. Poiché viene invocata sul numero di componenti attive, il numero di elementi scambiati decrescre ad ogni iterazione, in particolare nel caso peggiore sappiamo che le componenti dimezzano, pertanto la complessità è data da:

$$O(\sum_{i=0}^{\log V} \frac{V}{2^i} \log p) = O(V \log p \sum_{i=0}^{\log V} (\frac{1}{2})^i) = O(V \log p)$$

3.4 Funzione di isoefficienza

Tempo sequenziale: $O(E \log V)$.

Tempo parallelo: $O(\frac{E}{p} \log V + V \log V)$.

Complessità comunicazioni: $O(V \log p)$.

Overhead parallelo: $O(p(\frac{E}{p} \log V + V \log V + V \log p) - E \log V) = O(pV \log V + Vp \log p)$.

Relazione isoefficienza: Poiché l'overhead parallelo coinvolge più termini, bisogna trovare la relazione di isoefficienza per ciascuno di essi e considerare il termine asintoticamente maggiore, in particolare consideriamo il caso di grafi densi in cui $O(E) = O(V^2)$:

1. $E \log V \geq CpV \log V \Rightarrow V \geq Cp$ scalabilità ideale.
2. $E \log V \geq CVp \log p \Rightarrow V \log V \geq Cp \log p$ ovvero stessa condizione del primo termine.

Per grafi sparsi, dove $O(E) = O(V)$, è facile vedere che non si ottiene scalabilità, questo è dovuto dal fatto che il lavoro svolto da ciascun processore sugli archi, viene dominato dalla comunicazione, in quanto vengono comunicati tutti e V i vertici per sincronizzare la *UnionFind* e dal lavoro di *mapping* e di fusione delle componenti. Infatti, aumentando il numero di processori, $\frac{E}{p}$ diviene via via trascurabile in confronto a V quando $O(E) = O(V)$, si conclude che si ottiene ottima scalabilità solo per grafi densi.

Funzione di scalabilità: $M(V, E) = O(V + E)$, considerando grafi densi abbiamo $M(V, E) = O(V^2)$, pertanto $\frac{M(f(p))}{p} = \frac{C^2p^2}{p} = C^2p$ scalabilità accettabile solo per alcuni valori di p.
Nel caso di grafi sparsi non si ottiene scalabilità.

3.5 Benchmark e Speedup

Per l'analisi dell'implementazione sono stati generati grafi di diverse dimensioni con il seguente metodo[6]:

```
// n = numero di vertici
igraph_real_t p = 2 * log(n) / n;
error = igraph_erdos_renyi_game_gnp(&graph, n, p,
   IGRAPH_UNDIRECTED, IGRAPH_SIMPLE_SW,
   IGRAPH_EDGE_UNLABELED);
```

Il valore di p rende molto probabile che il grafo finale sia connesso, in caso contrario, un errore viene restituito, pertanto tutti i grafi generati hanno la garanzia di essere connessi. Per verificare la correttezza, è stato utilizzato l'algoritmo di Prim sequenziale. I risultati sono ottenuti su una media di cento iterazioni per ciascun scenario.

	V = 1000 E = 6753	V = 10000 E = 92201	V = 100000 E = 1150788
p = 1	0.000248s	0.003505s	0.051609s
p = 2	0.000229s V,E andrebbero aumentati	0.002662s V,E andrebbero aumentati	0.024768s Speedup Ideale
p = 3	0.000320s CO domina	0.002456s V,E andrebbero aumentati	0.021331s Speedup sublineare
p = 4	0.000460s CO domina	0.002699s V,E andrebbero aumentati	0.018317s Speedup sublineare

Tabella 1: Analisi dei tempi d'esecuzione per diversi ordini di V ed E su grafi Erdos Renyi senza mescolamento degli archi.

Analizzandoli, è evidente che la comunicazione domina per poche migliaia di archi e di vertici, ma si ottiene un buon beneficio quando si aumenta di un ordine di grandezza. Eseguendo una media di cento iterazioni su un grafo con 200'000 vertici e 2'441'945 di archi, su un singolo processore si impiega $\sim$0.117s, mentre su quattro processori si ha $\sim$0.0377s, pertanto si ottiene uno speedup di $\sim$3.11 e un'efficienza relativa del 78%. L'implementazione proposta risulta inoltre, molto sensibile all'*input*, i precedenti risultati sono ottenuti su esecuzioni con grafi generati casualmente, questi presentano elevata correlazione tra gli archi poiché quest'ultimi sono in ordine di creazione all'interno del file, di conseguenza si beneficia di maggior località dei dati, in quanto archi molto vicini nel grafo tendono ad apparire vicini anche nel file. Ciò rende l'esecuzione più rapida. Per simulare un ordinamento casuale degli archi ed analizzare le prestazioni in assenza di questa correlazione data dal generatore di grafi, è stato utilizzato un algoritmo di mescolamento[7] sul file in *input* che modifica solo l'ordine degli archi, mantenendo il grafo immutato. Successivamente sono stati ripetuti i *benchmarks*.

	V = 1000 E = 6753	V = 10000 E = 92201	V = 100000 E = 1150788
p = 1	0.000298s	0.004511s	0.075610s
p = 2	0.000275s V,E andrebbero aumentati	0.002706s Speedup sublineare	0.033716s Speedup super-lineare
p = 3	0.000307s CO domina	0.002937s V,E andrebbero aumentati	0.023810s Speedup super-lineare
p = 4	0.000416s CO domina	0.002951s V,E andrebbero aumentati	0.020585s Speedup lineare

Tabella 2: Analisi dei tempi d'esecuzione per diversi ordini di V ed E su grafi Erdos Renyi con mescolamento degli archi.

I risultati evidenziano come le prestazioni in termini di tempo d'esecuzione assoluto peggiorino, ma l'algoritmo risulta abbastanza robusto. Il peggioramento lo si ha per lo più sul singolo processore, mentre i tempi d'esecuzione paralleli, sebbene anch'essi peggiorati, rimangono simili allo scenario precedente. In questo specifico contesto, il beneficio dato dal calcolo parallelo è maggiore confrontato con lo scenario precedente, poiché il lavoro sequenziale è molto più sensibile all'*input*, di conseguenza le prestazioni sequenziali degradano, mentre quelle parallele rimangono abbastanza simili.

3.6 Metriche di Amdahl e Gustafson

Suddividendo i due scenari in *Ordinati* e *Mescolati* per indicare rispettivamente, il caso in cui l'input presenti il grafo con una forte correlazione tra gli archi e il caso in cui l'input non abbia un ordinamento preciso, si ottengono i seguenti valori di *Speedup* teorici con Gustafson e Amdahl.

Ordinati: Utilizzando 4 processori, 0.002507s sono spesi eseguendo codice sequenziale su un totale di 0.018317s, di conseguenza 13.69% è speso per codice sequenziale, pertanto lo *speedup* scalato secondo la legge di Gustafson è il seguente:

$$Speedup\ scalato \leq s + (1 - s)p = 0.1369 + (1 - 0.1369)4 = 3.59$$

Su un singolo processore la percentuale è pari a 4.86% ($0.002507s$ su $0.051609s$) pertanto lo *speedup* predetto dalla legge di Amdahl è:

$$Speedup \leq \frac{1}{f + \frac{1-f}{p}} = \frac{1}{0.0486 + \frac{1-0.0486}{4}} = 3.49$$

Mescolati: Utilizzando 4 processori, 0.002521s sono spesi eseguendo codice sequenziale su un totale di 0.020585s, di conseguenza 12.25% è speso per codice sequenziale, pertanto lo *speedup* scalato secondo la legge di Gustafson è il seguente:

$$\textit{Speedup scalato} \leq s + (1-s)p = 0.1225 + (1 - 0.1225)4 = 3.63$$

Su un singolo processore la percentuale è pari a 3.33% ($0.002521s$ su $0.075610s$) pertanto lo *speedup* predetto dalla legge di Amdahl è:

$$\textit{Speedup} \leq \frac{1}{f + \frac{1-f}{p}} = \frac{1}{0.0333 + \frac{1-0.0333}{4}} = 3.63$$

3.7 Specifiche PC

Tutti i *benchmarks* sono stati svolti su MacBook Air M2 2022 con 4 Performance Cores.

Riferimenti bibliografici

[1] University of Illinois, *CS 374 AL1 - Algorithms for Minimum Spanning Trees*, `https://courses.grainger.illinois.edu/cs374al1/fa2024/lec/slides/21_mst.pdf`

[2] GeeksforGeeks, *Borůvka's Algorithm - Greedy Algo-9*, `https://www.geeksforgeeks.org/dsa/boruvkas-algorithm-greedy-algo-9/`

[3] Wikipedia, *Borůvka's algorithm*, `https://en.wikipedia.org/wiki/Bor%C5%AFvka%27s_algorithm`

[4] GeeksforGeeks, *Introduction to Disjoint Set Data Structure (Union-Find Algorithm)*, `https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/`

[5] CP-Algorithms, *Disjoint Set Union*, `https://cp-algorithms.com/data_structures/disjoint_set_union.html`

[6] Mathematics Stack Exchange, *Connectivity of a graph in the Erdős–Rényi model*, `https://math.stackexchange.com/questions/4087765/connectivity-of-a-graph-in-the-erd%C5%91s-r%C3%A9nyi-model`

[7] Wikipedia, *Fisher–Yates shuffle*, `https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle`